1. Load the Original Event Tables Directly

Notebook cells: Import libraries -> Load the original event tables directly

LOAD

6 source CSVs (raw files on disk)

IN

admissions.csv, patients.csv, ward_movements.csv, infection_events.csv, device_exposure.csv, medications.csv

DOES

Six separate, already-clean exports - one table per real-world entity. Unlike the old master.csv approach, nothing here has been pre-merged/inflated, so there is no reconstruction step needed at all.

OUT

6 raw DataFrames, loaded with pd.read_csv() - admissions, patients, ward_stays (from ward_movements.csv), infections (from infection_events.csv), devices (from device_exposure.csv), medications.

↓

CLEAN

Rename columns + attach patient_id

IN

ward_stays, devices, medications, admissions

DOES

ward_movements/device_exposure/medications each call their own timestamps just 'start_datetime'/'end_datetime' - renamed to ward_start/end_datetime, device_start/end_datetime and med_start/end_datetime so they don't clash once merged together. ward_movements.csv has no patient_id, so it's merged in from admissions.csv.

OUT

ward_stays, infections, devices, medications - now with exactly the column names the rest of the notebook expects (infections already matched, no rename needed).

↓

CLEAN

Parse timestamps - no placeholder rows to drop

IN

every *_datetime column, across all 5 tables

DOES

pd.to_datetime(errors='coerce', format='mixed') on each one, same as before. Unlike master.csv, device_exposure.csv and medications.csv contain ONLY real records - no 'No device'/'No antibiotic' placeholder rows - so there is nothing to filter out at this stage.

OUT

ward_stays, infections, devices, medications - same rows, timestamps now real datetime objects, ready for the next stage.

↓ continues on: Page 2 - Attribute infections to wards & build the ward-day panel

2. Attribute Infections to Wards & Build the Ward-Day Panel

Notebook cells: Attribute each infection to a ward -> Build the ward-day panel

↑ continued from: Page 1 - Load & clean the raw data

CLEAN

attribute_to_ward(infections, ward_stays)

IN
infections + ward_stays

DOES
Match each infection's timestamp to the one ward-stay window that contains it (ward_start_datetime <= infection_datetime < ward_end_datetime). Infections detected after discharge fall back to the admission's LAST ward stay.

OUT
inf_ward - same as infections, plus ward, ward_start_datetime, ward_end_datetime attached.

SAVED
infections_attributed_to_ward.csv

↓

BUILD

expand_stay_to_days() + groupby(ward, date)

IN
ward_stays

DOES
Explode every ward stay into one row per calendar day it overlaps, then count unique patients per ward per day.

OUT
census - ward, date, n_at_risk (1 row = 1 ward-day).

↓

BUILD

Device-overlap logic + expand_overlap_to_days() + groupby

IN
devices + ward_stays

DOES
Clip each device interval to the ward interval it overlaps with, explode that overlap into daily rows, count per ward-day (pooled, and split by device_type).

OUT
device_days - ward, date, device_days (+ per-device-type columns).

↓

BUILD

build_panel(inf_ward, census, device_days)

IN
inf_ward + census + device_days

DOES
Count new infections per ward-day; merge onto census + device_days; fill missing combinations with 0; compute device_utilisation = device_days / n_at_risk.

OUT
panel - THE central table for everything that follows. Columns: ward, date, n_at_risk, new_infections, device_days, device_utilisation (+ per-device-type columns). 1 row = 1 ward, 1 calendar day.

SAVED
ward_day_panel.csv

↓ continues on: Page 3 - Explore the panel & split into train / validation / test

3. Explore the Panel & Split Train / Validation / Test

Notebook cells: Exploratory plots -> Split the data into train / validation / test

↑ continued from: Page 2 - Attribute infections to wards & build the ward-day panel

BUILD

Exploratory plots

IN

panel

DOES

Line/point plots of n_at_risk, new_infections and device_utilisation over time, one facet per metric, coloured by ward - purely to look at the data before modelling.

OUT

panel is unchanged - this step only visualises it, no new columns or rows.

SPLIT

split_train_val_test(panel)

IN

panel (every ward, every day)

DOES

Sort the panel's unique calendar dates; cut chronologically into the earliest ~70% (train), next ~15% (validation), final ~15% (test) - NOT a random row split, because ward-days close in time are related, and a random split would leak information.

OUT

train_panel, val_panel, test_panel - exactly the same columns as panel; only the ROWS are divided, by date.

↓ continues on: Page 4 - Fit three candidate models on the training data

4. Fit Three Candidate Models (Training Data Only)

Notebook cells: Fit the Poisson model -> Fit three models, on the training data only

↑ continued from: Page 3 - Explore the panel & split train / validation / test

MODEL

fit_three_models(train_panel)

IN

train_panel only (validation and test are not touched here)

DOES

offset = log(n_at_risk). Fit 3 Poisson GLMs with statsmodels (smf.glm, family=Poisson - the Python equivalent of R's glm(family=poisson)): Model 1: new_infections ~ 1. Model 2: + device_utilisation. Model 3: + C(ward).

OUT

3 fitted statsmodels GLM result objects (one per model).



MODEL

summarise_model() x3

IN

a fitted GLM result object

DOES

Pull out coefficients, standard errors, z, p-values, 95% CI; exponentiate to IRR; clean ward dummy names (e.g. 'C(ward)[T.ICU]' -> 'ICU'); compute dispersion (Pearson chi2 / df_resid).

OUT

res1, res2, res3 - each a dict with: name, aic, deviance, dispersion, the fitted model object itself, and table (1 row per predictor: term, coef, se, z, p_value, sig, IRR, IRR_2.5%, IRR_97.5%).
comparison - 1 row per model: model, AIC, dispersion.

↓ continues on: Page 5 - Pick the best model using the validation set

5. Pick the Best Model Using the Validation Set

Notebook cells: Pick the best model -> Save comparison/coefficients -> Check dispersion

↑ continued from: Page 4 - Fit three candidate models on the training data

EVALUATE

attach_predictions(val_panel, model) x3

IN
val_panel + one fitted model

DOES
Ask the model to predict on validation days it never trained on.

OUT
val_panel + predicted (continuous) + predicted_infections (rounded whole number).

↓

EVALUATE

evaluate_on() x3 + pick lowest RMSE

IN
predictions on val_panel, for each of the 3 models

DOES
Compute MAE/RMSE per model on validation data; merge onto comparison as val_MAE/val_RMSE; the model with the LOWEST validation RMSE is selected - not just lowest training AIC, since a complex model can always fit training data better.

OUT
best - the winning model's res dict.
comparison - now has val_MAE, val_RMSE columns too.

SAVED
model_comparison.csv, best_model_coefficients.csv, model1/2/3_*_coefficients.csv

↓

EVALUATE

Dispersion check on `best`

IN
best['dispersion'] (mean vs variance ratio, from training data)

DOES
If far from 1 (outside [0.8, 1.2]): widen standard errors/CI by sqrt(dispersion) - a standard 'quasi-Poisson' adjustment.

OUT
adjusted - same rows/terms as best['table'], wider se/CI if needed.

SAVED
best_model_coefficients_quasipoisson.csv (if adjustment applied), dispersion_by_ward.csv

↓ continues on: Page 6 - Evaluate the chosen model on the held-out test set

6. Evaluate the Chosen Model on the Held-Out Test Set

Notebook cells: Predicted vs actual - the honest check, on data the model has never seen

↑ continued from: Page 5 - Pick the best model using the validation set

EVALUATE

attach_predictions(test_panel, best)

IN
test_panel (touched for the very first time here)
DOES
Predict on test days - never used for fitting OR for choosing the model.
OUT
test_with_predictions = test_panel + predicted + predicted_infections.

↓

EVALUATE

Print actual vs predicted table + plot_predicted_vs_actual()

IN
test_with_predictions
DOES
Print a table of new_infections next to predicted_infections (the discrete, rounded whole-number version - real infection counts can't be fractional). Plot both as dots over time, one panel per ward - dots only, no connecting lines.
OUT
A printed comparison table + a saved PNG figure; no new columns.
SAVED
pooled_test_predicted_vs_actual.png

↓

EVALUATE

plot_residuals()

IN
test_with_predictions
DOES
pearson_resid = (actual - predicted) / sqrt(predicted); scatter over time by ward - should hover randomly around 0 if the model isn't systematically off.
OUT
test_with_predictions + pearson_resid column.
SAVED
pooled_test_residuals.png

↓

EVALUATE

evaluate_on() across train / validation / test

IN
train_panel, val_panel, test_panel + best model
DOES
MAE/RMSE on all three splits, side by side - train error is expected lowest; test error is the trustworthy, out-of-sample number.
OUT
all_scores - 1 row per split: split, model, mae, rmse, n.
SAVED
panel_pooled_with_predictions.csv (train+val+test, with a 'split' column), pooled_train_val_test_scores.csv

↓ continues on: Page 7 - Repeat the whole pipeline per organism + drug combination

7. Repeat the Whole Pipeline per Organism + Drug Combination

Notebook cells: Drug-bug specific analysis -> Which combination has the most data? -> Run it

↑ continued from: Page 6 - Evaluate the chosen model on the held-out test set

CLEAN

Count infections by organism x drug

IN
infections + medications

DOES
Merge, group by (organism, drug), count, sort - to find which combination has enough data to model (e.g. the largest one, and E. coli + Ceftriaxone specifically).

OUT
combo_counts - 1 row per organism+drug pair: organism, drug, n_infections.

SAVED
organism_drug_combination_counts.csv



CLEAN

build_drug_bug_infections(organism, drug)

IN
infections + medications

DOES
Keep only infections of this organism, in admissions that received this drug at some point in their stay.

OUT
inf_subset - same columns as infections, far fewer rows.



BUILD

run_drug_bug_analysis() = Pages 2-6, replayed on inf_subset

IN
inf_subset

DOES
attribute_to_ward -> build_panel -> split_train_val_test -> fit_three_models -> pick best on validation -> attach_predictions on test -> plot -> evaluate. Exactly the same steps as the pooled analysis, just starting from a smaller, filtered set of infections.

OUT
panel_sub (ward-day panel for this combo only), best_sub (chosen model), test_with_pred_sub (+predicted, +predicted_infections), test_score_sub (mae/rmse/n).
Run once for the largest combination, once for E. coli + Ceftriaxone.

SAVED
largest_combo_*.csv, ecoli_ceftriaxone_*.csv (panel-with-predictions + coefficients)

↓ continues on: Page 8 - Build the final results-interpretation PDFs

8. Build the Final Results-Interpretation PDFs

Notebook cells: Results interpretation PDFs -> make_summary_pdf() x3

↑ continued from: Page 7 - Repeat the whole pipeline per organism + drug combination

OUTPUT

make_summary_pdf() - called once per analysis

IN

For each of: (a) the pooled analysis, (b) the largest combo, (c) E. coli + Ceftriaxone - its full panel, train/val/test panels, model comparison table, chosen model & coefficients, test-set predictions, test-set score, and (pooled only) the exploratory plot.

DOES

Assembles a ready-to-read PDF: what data was used -> train/val/test split -> model comparison -> chosen model's coefficients/IRRs -> predicted-vs-actual on the test set -> residuals on the test set -> one-paragraph summary.

OUT

3 separate PDF files, one per analysis.

SAVED

Poisson_HAI_Results_Summary_Pooled.pdf
Poisson_HAI_Results_Summary_LargestCombo.pdf
Poisson_HAI_Results_Summary_EcoliCeftriaxone.pdf

OUTPUT

End of pipeline

IN

-

DOES

Every stage above has now run once for the pooled data and once per organism+drug combination.

OUT

All CSVs, PNGs and PDFs listed across these 8 pages are sitting in the notebook's working folder, ready to be pulled into the thesis.